

Azure Data Engineering – Class Notes

Managed vs External Tables • Medallion Architecture • Azure Data Engineering Architecture • Databricks Environments

1. Types of Tables

In a data platform such as Azure Databricks with Unity Catalog, tables can broadly be classified into two types:

Managed Tables and **External Tables**. The main difference is who manages the physical data and where that data is stored.

1.1 Managed Table

A **managed table** is a table where the data platform manages both the table metadata and the underlying data. The table is created inside a catalog and schema, and the platform determines the appropriate storage location.

```
Catalog
↓
citibike_catalog
↓
Schema
↓
citibike_schema
↓
Table
↓
citibike_trips
```

Example:

```
df.write \
  .format("delta") \
  .mode("overwrite") \
  .saveAsTable("citibike_catalog.citibike_schema.citibike_trips")
```

Dropping a managed table:

```
DROP TABLE citibike_catalog.citibike_schema.citibike_trips;
```

The table metadata is removed and the underlying data is generally deleted as well. The exact behavior can depend on the platform and catalog implementation, but this is the standard distinction.

1.2 External Table

An **external table** is a table where the table metadata is managed by the platform, but the physical data is stored at an externally specified location. This is useful when data already exists in ADLS Gen2 and you want to register that data as a table.

```
ADLS Gen2
■
■■■ citibike/
    ■■■ bronze/
        ■■■ data1.parquet
        ■■■ data2.parquet
        ■■■ data3.parquet
```

```
CREATE TABLE citibike_catalog.citibike_schema.citibike_bronze
USING DELTA
LOCATION 'abfss://container@storageaccount.dfs.core.windows.net/citibike/bronze';
```

Dropping an external table removes the table registration/metadata, but the underlying data remains in the external storage location.

Aspect	Managed Table	External Table
Metadata	Managed by platform	Managed by platform
Data	Managed by platform	Stored at specified external location

Aspect	Managed Table	External Table
Storage path	Platform-managed	User/platform specifies location
DROP TABLE	Generally removes data too	Removes metadata; data remains
Typical use	Platform-managed datasets	Existing/shared data in ADLS

Memory trick: Managed = Databricks manages the data. External = data lives at an externally specified location.

2. Medallion Architecture

The **Medallion Architecture** organizes data into levels of refinement. The basic flow is:

RAW → BRONZE → SILVER → GOLD

2.1 RAW Layer

The **RAW layer** contains the original/source data, with minimal or no transformation. Keeping source data provides a reference point if downstream processing needs to be recreated or investigated.

Source System
↓
RAW

Example source record:

```
{
  "customer_id": 101,
  "name": "Abhi",
  "city": "Mysuru",
  "age": 25
}
```

2.2 BRONZE Layer

The **Bronze layer** contains raw data ingested into the data platform with minimal transformation. Typical operations include ingestion timestamps, source information, and other technical metadata.

API / CSV / Database
↓
Bronze

```
customer_id | name | city | age | ingestion_time
101         | Abhi | Mysuru | 25 | 2026-10-04 10:00:00
```

2.3 SILVER Layer

The **Silver layer** contains cleaned, validated, and transformed data. Typical operations include:

- Removing duplicates
- Handling NULL values
- Correcting data types
- Standardizing values
- Joining datasets
- Applying business rules
- Data validation

Example:

Bronze:

```
customer_id | name | city | age
101         | Abhi | Mysuru | 25
102         |      | mysore | 26
101         | Abhi | Mysuru | 25
```

```

Silver:
customer_id | name      | city   | age
101         | Abhi     | Mysuru | 25
102         | Unknown  | Mysuru | 26

```

The duplicate is removed, **mysore** is standardized to **Mysuru**, and the missing name is handled.

2.4 GOLD Layer

The **Gold layer** contains business-ready, aggregated, and optimized data for reporting and analytics.

```

city          | total_customers | total_revenue
Mysuru        | 15,000           | ■25,00,000
Bengaluru     | 40,000           | ■85,00,000

```



Layer	Purpose	Typical activity
RAW	Original/source data	Preserve source data
BRONZE	Ingested raw data	Minimal transformation + technical metadata
SILVER	Cleaned/transformed data	Validation, deduplication, joins, business rules
GOLD	Business-ready data	Aggregations, reporting, analytics

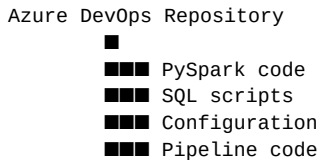
Memory trick: RAW = Original; BRONZE = Ingested; SILVER = Cleaned; GOLD = Business-ready.

3. Azure Data Engineering Architecture

A typical Azure Data Engineering environment combines source control, orchestration, processing, storage, security, authentication, and governance. Each service has a specific responsibility.

3.1 Azure DevOps — Repository / Source Control

Azure DevOps can be used for source code management and development workflows. It stores source code, maintains version history, and supports branching and pull requests.



- Source code repositories
- Version history
- Branching
- Pull requests
- Collaboration

3.2 Azure Databricks — Processing & Job Execution

Azure Databricks runs PySpark, Spark SQL, Python, SQL, and data engineering workloads. It supports notebooks, jobs, scheduling, and transformations.

```

ADLS Gen2
  ↓
Databricks
  ↓
PySpark transformation
  ↓
Delta table

```

- Develop notebooks
- Run Spark workloads
- Create and execute jobs
- Schedule jobs
- Transform large datasets

3.3 Azure ADLS Gen2 — Data Storage

ADLS Gen2 is the storage layer for the data platform. It can store Parquet, Delta, ORC, CSV, and JSON.

```

ADLS Gen2
■
■■■ raw/
■■■ bronze/
■■■ silver/
■■■ gold/

```

- Parquet
- Delta
- ORC
- CSV
- JSON

3.4 Azure SQL Database — Relational Database

Azure SQL Database stores structured relational data in tables, rows, and columns. It can support application or analytical workloads depending on the architecture.

```

Customer
-----
customer_id
customer_name
email
city

```

- Application data
- Structured relational workloads
- Analytical workloads depending on architecture

3.5 Azure App Registration — Application Identity & Authentication

Azure App Registration creates an application identity/service principal that applications and data pipelines can use to authenticate to Azure resources.

```

Application
  ↓
Service Principal
  ↓
Azure Authentication
  ↓
Azure Resources

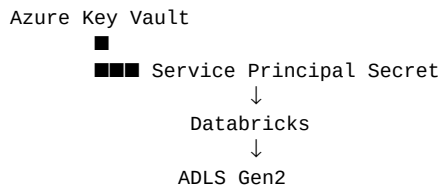
```

- Application identity

- Service principal
- Authentication to Azure resources

3.6 Azure Key Vault — Secrets & Credential Management

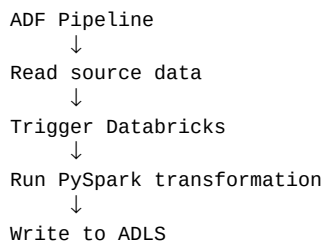
Azure Key Vault securely stores passwords, secrets, keys, and certificates and helps avoid hardcoding credentials in notebooks or source code.



- Passwords
- Secrets
- Keys
- Certificates

3.7 Azure Data Factory — Data Integration & Orchestration

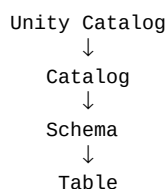
ADF creates and orchestrates data pipelines, supports scheduling and triggering, moves data, coordinates Azure services, and can trigger Databricks jobs/notebooks.



- Create pipelines
- Schedule pipelines
- Trigger workflows
- Move data
- Coordinate Azure services

3.8 Unity Catalog — Data & AI Governance

Unity Catalog is Databricks' centralized governance solution for data and AI assets. It provides metadata management, access permissions, lineage, and governance.



- Metadata management
- Access permissions
- Data lineage
- Governance of data and AI assets

Unity Catalog example

```
citibike_catalog
  ↓
citibike_schema
  ↓
citibike_bronze
citibike_silver
citibike_gold
```

4. Databricks Workspaces / Environments

Professional data engineering projects commonly separate environments so development and testing do not directly interfere with production workloads.

DEV → TEST → PROD

4.1 DEV — Development

DEV is where developers build and modify code: PySpark notebooks, transformations, debugging, and initial pipelines.

```
Developer
  ↓
DEV Databricks
  ↓
Develop PySpark pipeline
```

4.2 TEST — Testing

TEST is used for integration and validation. Code promoted from DEV is tested with realistic data and connected components.

```
DEV
  ↓
Code promotion
  ↓
TEST
```

Typical validation questions:

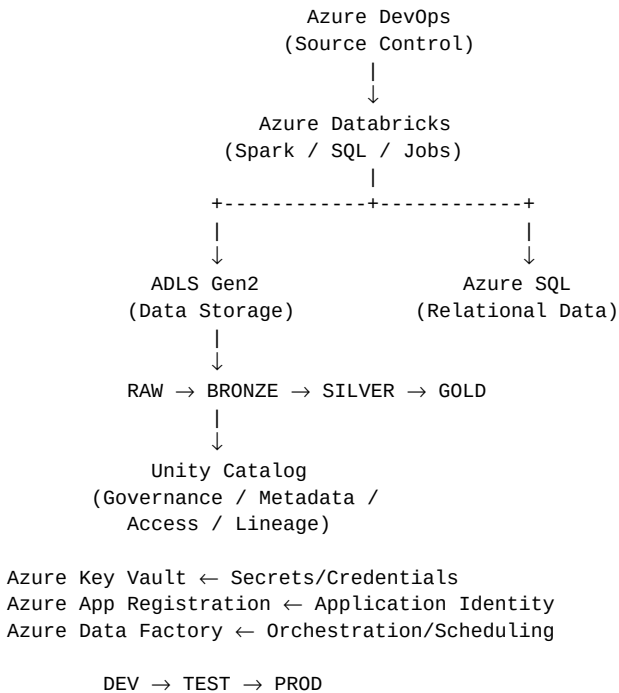
- Does the pipeline work correctly?
- Does the transformation produce the expected result?
- Does it work with realistic data?
- Do different components integrate correctly?

4.3 PROD — Production

PROD is the live environment containing production-grade data, pipelines, jobs, and workloads. Developers generally should not experiment directly with production workloads.

```
DEV
  ↓
Development
  ↓
TEST
  ↓
Validation
  ↓
PROD
  ↓
Production execution
```

Overall Azure Data Engineering Architecture



Practical Example — Citibike Data Pipeline

This example connects the services and architecture concepts using a Citibike-style data engineering workflow.

Step 1 — Source data

Citibike provides source data in formats such as CSV or JSON.

Step 2 — Azure Data Factory

ADF orchestrates the workflow by triggering the pipeline according to the required schedule or event.

Step 3 — ADLS Gen2

Raw source data is stored in ADLS Gen2, for example under a raw/citibike location.

Step 4 — Azure Databricks

Databricks reads the source data and performs transformations using PySpark.

Step 5 — Medallion Architecture

Data progresses through RAW → BRONZE → SILVER → GOLD. Bronze contains minimally transformed ingested data, Silver contains cleaned and validated data, and Gold contains business-ready datasets.

Step 6 — Unity Catalog

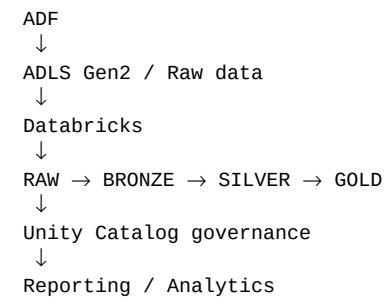
Tables can be registered and governed through Unity Catalog, with catalogs, schemas, permissions, metadata, and lineage.

Step 7 — Key Vault + App Registration

Secrets and credentials are protected using Key Vault, while application identity/service principals support authentication.

Step 8 — DEV → TEST → PROD

The developed pipeline is promoted through environments, with source code maintained and reviewed through Azure DevOps.



Quick Interview Revision

Component	Main responsibility
Azure DevOps	Source control & collaboration
Databricks	Data processing & Spark jobs
ADLS Gen2	Data storage
Azure SQL	Relational/structured data
ADF	Data integration & orchestration
App Registration	Application identity
Key Vault	Secrets & credentials
Unity Catalog	Governance, metadata, access & lineage
RAW	Original/source data
BRONZE	Ingested raw data
SILVER	Cleaned/transformed data
GOLD	Business-ready data
DEV	Development
TEST	Validation/integration testing
PROD	Production workloads

Final memory picture: Azure DevOps = source control • ADF = orchestration • Databricks = processing • ADLS Gen2 = storage • Unity Catalog = governance • Key Vault = secrets • App Registration = application identity • RAW → BRONZE → SILVER → GOLD = data refinement • DEV → TEST → PROD = environment promotion.